

Industrialisation d'un Pipeline E-commerce

Git · Jenkins · Apache Airflow · MongoDB · Docker

Master Informatique / Data Engineering

Auteur : habib-Rouabah **Dépôt :** <https://github.com/habib-Rouabah/airflow-ecommerce-project>

Date : 4 octobre 2026

Sommaire

1. Contexte métier
 2. Problématique
 3. Dataset utilisé
 4. Architecture technique
 5. Description du DAG Airflow
 6. Pipeline CI/CD Jenkins
 7. Stockage MongoDB
 8. Résultats obtenus
 9. Captures d'écran
 10. Difficultés rencontrées
 11. Conclusion
-

1. Contexte métier

Une entreprise de commerce électronique spécialisée dans la vente de produits informatiques souhaite moderniser son système d'analyse des ventes. Aujourd'hui, les données sont exportées quotidiennement sous forme de fichiers CSV et traitées **manuellement** par les équipes métier.

Cette approche présente plusieurs limites :

- production tardive des indicateurs ;
- risque d'erreurs humaines ;
- difficulté à suivre l'évolution des ventes ;
- absence d'historisation des traitements ;
- faible traçabilité des opérations réalisées.

L'objectif du projet est de mettre en place une **plateforme automatisée** capable de collecter, valider, transformer, analyser et stocker les données de ventes afin de produire des indicateurs décisionnels fiables, de façon reproductible et tracée.

2. Problématique

Comment industrialiser le traitement quotidien des ventes e-commerce pour garantir des indicateurs fiables, historisés et produits automatiquement, tout en assurant la qualité des données et la traçabilité des exécutions ?

Les enjeux techniques sont les suivants :

- **Automatisation** : supprimer les traitements manuels (orchestration Airflow).
- **Qualité** : détecter et isoler les lignes invalides avant calcul.
- **Fiabilité** : un calcul déterministe et reproductible des indicateurs.
- **Industrialisation** : intégration et déploiement continu (Jenkins), tests automatisés, conteneurisation (Docker).
- **Historisation** : conserver chaque exécution et son statut dans MongoDB.

3. Dataset utilisé

Le projet cible le dataset **Olist Brazilian E-Commerce**

(<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>), représentatif d'un e-commerce réel (> 100 000 commandes, clients, produits, paiements, régions).

Pour rendre le pipeline **exécutable et testable sans téléchargement**, un générateur de données synthétiques respectant le même schéma est fourni (`scripts/generate_dataset.py`). Le dataset de démonstration contient **20 000 lignes**.

Schéma minimal (`data/dataset.csv`) :

Champ	Description
CommandeID	Identifiant unique de la commande
Date	Date de la transaction
Produit	Produit vendu
Categorie	Catégorie du produit

Champ	Description
Quantite	Quantité vendue
Prix	Prix unitaire
Montant	Montant total de la vente
Region	Région / ville du client
Client	Identifiant du client

4. Architecture technique



Les composants sont **conteneurisés** via `docker-compose.yml` :

Service	Rôle	Port
PostgreSQL	Métadonnées Airflow	interne
MongoDB	Stockage des indicateurs	27017
Airflow Webserver	Interface / supervision des DAGs	8080
Airflow Scheduler	Ordonnancement des tâches	—
Jenkins	Intégration et déploiement continus	8081

5. Description du DAG Airflow

Le DAG `ecommerce_sales_pipeline` (planifié `@daily`) enchaîne les tâches suivantes :

#	Tâche	Rôle
1	<code>attendre_fichier</code>	<code>FileSensor</code> : attend la présence du CSV
2	<code>verifier_existence_fichier</code>	échoue si le fichier est absent
3	<code>verifier_fichier_non_vide</code>	arrêt propre (skip) si fichier vide
4	<code>controler_qualite</code>	sépare lignes valides/rejetées → <code>errors.csv</code> , pousse les compteurs en XCom
5	<code>choisir_chemin</code>	<code>BranchPythonOperator</code> : traite ou arrête selon le taux de validité
6	<code>charger_donnees</code>	charge et liste les catégories (XCom)
7	<code>calculer_kpis</code>	calcule tous les indicateurs, document en XCom
8	<code>analyse_categorie_*</code>	tâches créées dynamiquement , une par catégorie
9	<code>generer_rapport</code>	rapport final + statut (<code>ALL_DONE</code>)
10	<code>stocker_mongodb</code>	insertion du document dans MongoDB

Points techniques mis en œuvre (cf. cahier des charges §6.2) :

- **FileSensor** en mode `reschedule` (libère le worker entre deux sondages) ;
- **XComs** pour transmettre métriques et catégories entre tâches ;
- **BranchPythonOperator** pour l'aiguillage conditionnel ;
- **Génération dynamique** de tâches d'analyse par catégorie ;

- **Trigger Rules** (`ALL_DONE` , `NONE_FAILED` , `NONE_FAILED_MIN_ONE_SUCCESS`) pour la gestion des erreurs et la production d'un statut `success` / `partial` / `failed` .

Règles de gestion appliquées (§8)

- identifiant de commande unique (doublons rejetés) ;
- montant négatif → ligne rejetée ;
- quantité nulle ou négative → ligne invalide ;
- fichier vide → arrêt propre du workflow ;
- lignes incorrectes isolées dans `errors.csv` ;
- chaque exécution historisée dans MongoDB avec son statut.

6. Pipeline CI/CD Jenkins

Le `Jenkinsfile` déclare 7 stages :

1. **Checkout** — récupération du code depuis Git ;
2. **Install dependencies** — création d'un venv et installation de `requirements.txt` ;
3. **Run tests** — exécution de `pytest` (rapport JUnit) ;
4. **Validate DAG** — `python -m py_compile dags/*.py` ;
5. **Deploy DAG** — copie des DAGs vers le dossier Airflow ;
6. **Trigger DAG** — `airflow dags trigger ecommerce_sales_pipeline` ;
7. **Verify MongoDB** — `scripts/check_mongodb.py` vérifie l'insertion.

La suite de tests unitaires (`tests/test_pipeline.py`) couvre la validation des lignes, le calcul des métriques globales, le top produits, les agrégations par région et catégorie, l'évolution mensuelle et la structure du document final — **8 tests, tous verts**.

7. Stockage MongoDB

- **Base** : `ecommerce_analytics`
- **Collection** : `sales_metrics`

Un document est inséré par exécution, structuré ainsi :

```
{
  "execution_date": "2026-10-04",
  "dag_id": "ecommerce_sales_pipeline",
  "dataset": "olist",
  "source_file": "dataset.csv",
  "status": "partial",
```

```

"global_metrics": { "nb_commandes": 19606, "nb_clients": 3973,
                    "chiffre_affaires": 8937482.0, "panier_moyen": 455.85 },
"top_products":    [ ... ],
"category_metrics": [ ... ],
"region_metrics":  [ ... ],
"monthly_evolution": [ ... ],
"quality": { "valid_rows": 19606, "invalid_rows": 394, "error_file":
            "errors.csv" }
}

```

8. Résultats obtenus

Exécution sur le dataset de démonstration (20 000 lignes) :

Qualité des données

Indicateur	Valeur
Lignes totales	20 000
Lignes valides	19 606
Lignes rejetées	394
Taux de validité	98,0 %

Les 394 lignes rejetées correspondent aux montants négatifs, quantités nulles et identifiants de commande dupliqués injectés volontairement pour éprouver les règles de qualité.

Indicateurs globaux

Indicateur	Valeur
Nombre de commandes	19 606
Nombre de clients	3 973
Chiffre d'affaires	8 937 482 €
Panier moyen	455,85 €

Top 3 produits (par quantité vendue)

Produit	Quantité	CA (€)
Clavier	2 639	79 170
Ramette	2 600	20 800
Volant	2 583	516 600

Chiffre d'affaires par catégorie

Catégorie	Quantité	CA (€)
informatique	9 701	4 002 050
telephonie	7 498	1 641 940
gaming	7 327	1 267 690
audio	7 368	593 250
reseau	7 392	516 762
bureautique	4 998	332 540
electromenager	4 987	324 830
accessoires	9 831	258 420

Chiffre d'affaires par région (top 3)

Région	Commandes	CA (€)
Belo Horizonte	2 885	1 326 194
Rio	2 815	1 324 825
Salvador	2 877	1 319 488

L'évolution des ventes est répartie sur **10 mois** (janvier à octobre 2026).

9. Captures d'écran

9.1 Airflow — DAG en succès

Exécution `demo_run_1` du DAG `ecommerce_sales_pipeline` : toutes les tâches en succès (la tâche `arreter_pipeline` est `skipped`, branche non empruntée).



DAG Airflow en succès

9.2 MongoDB — document inséré

Document stocké dans la collection `ecommerce_analytics.sales_metrics` (indicateurs globaux et top produits).

```

airflow-ecommerce-project git:(main) docker compose exec mongodbd mongosh ecommerce_analytics --eval 'db.sales_metrics.
find({}, {_id:0}).sort({_id:-1}).limit(1).pretty()'
[
  {
    execution_date: '2026-10-04',
    dag_id: 'ecommerce_sales_pipeline',
    dataset: 'olist',
    source_file: 'dataset.csv',
    status: 'partial',
    global_metrics: {
      nb_commandes: 19606,
      nb_clients: 3973,
      chiffre_affaires: 8937482,
      panier_moyen: 455.85
    },
    top_products: [
      { product: 'Clavier', sales: 2639, revenue: 79170 },
      { product: 'Ramette', sales: 2600, revenue: 20800 },
      { product: 'Volant', sales: 2583, revenue: 516600 },
      { product: 'Smartphone', sales: 2572, revenue: 1543200 },
      { product: 'Routeur', sales: 2560, revenue: 217600 },
      { product: 'Enceinte', sales: 2515, revenue: 301800 },
      { product: 'Souris', sales: 2508, revenue: 50160 },
      { product: 'Cafetiere', sales: 2507, revenue: 225630 },
      { product: 'Disque SSD', sales: 2500, revenue: 200000 },
      { product: 'Chargeur', sales: 2485, revenue: 62125 }
    ],
    category_metrics: [
      { category: 'informatique', quantity: 9701, revenue: 4002050 },
      { category: 'telephonie', quantity: 7498, revenue: 1641940 },
      { category: 'gaming', quantity: 7327, revenue: 1267690 },
      { category: 'audio', quantity: 7368, revenue: 593250 },
      { category: 'reseau', quantity: 7392, revenue: 516762 },
      { category: 'bureautique', quantity: 4998, revenue: 332540 },
      { category: 'electromenager', quantity: 4987, revenue: 324830 },
      { category: 'accessoires', quantity: 9831, revenue: 258420 }
    ],
    region_metrics: [
      { region: 'Belo Horizonte', orders: 2885, revenue: 1326194 },
      { region: 'Rio', orders: 2815, revenue: 1324825 },
      { region: 'Salvador', orders: 2877, revenue: 1319488 },
      { region: 'Fortaleza', orders: 2808, revenue: 1284501 },
      { region: 'Curitiba', orders: 2774, revenue: 1247515 }
    ]
  }
]

```

Document MongoDB — métriques globales

Détail des agrégations par catégorie, par région, évolution mensuelle et qualité.

```

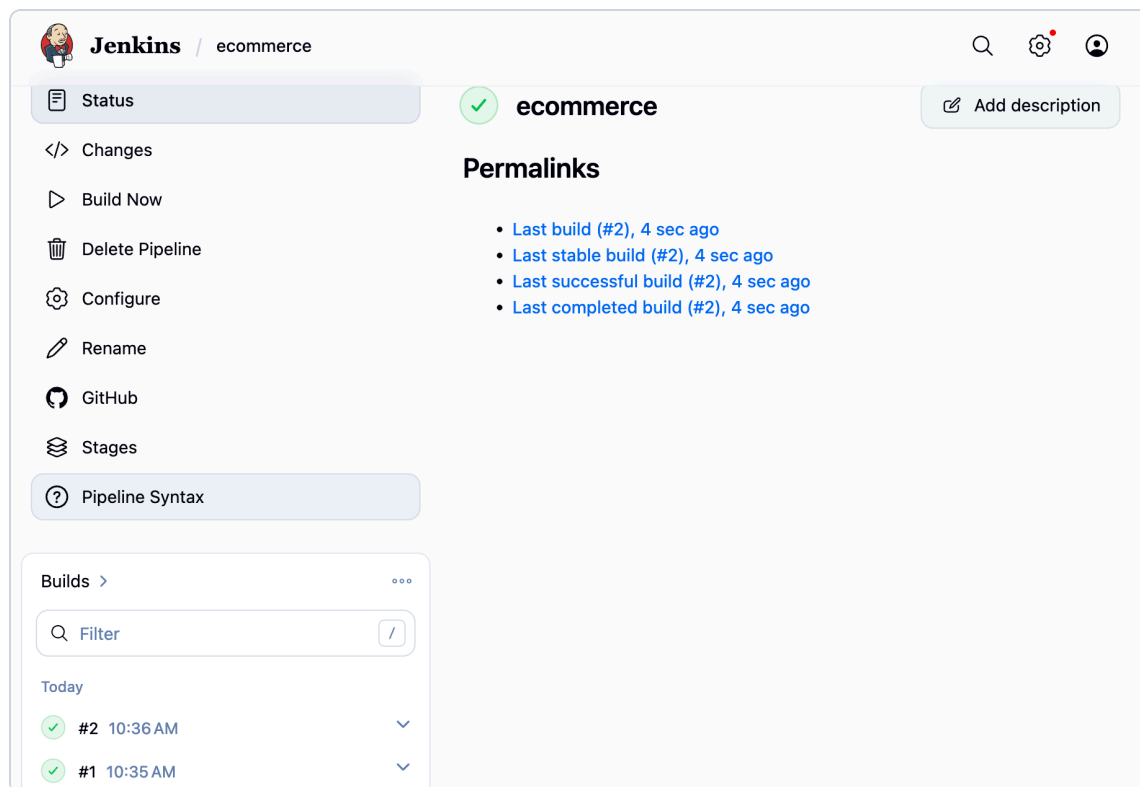
    ],
    category_metrics: [
      { category: 'informatique', quantity: 9701, revenue: 4002050 },
      { category: 'telephonie', quantity: 7498, revenue: 1641940 },
      { category: 'gaming', quantity: 7327, revenue: 1267690 },
      { category: 'audio', quantity: 7368, revenue: 593250 },
      { category: 'reseau', quantity: 7392, revenue: 516762 },
      { category: 'bureautique', quantity: 4998, revenue: 332540 },
      { category: 'electromenager', quantity: 4987, revenue: 324830 },
      { category: 'accessoires', quantity: 9831, revenue: 258420 }
    ],
    region_metrics: [
      { region: 'Belo Horizonte', orders: 2885, revenue: 1326194 },
      { region: 'Rio', orders: 2815, revenue: 1324825 },
      { region: 'Salvador', orders: 2877, revenue: 1319488 },
      { region: 'Fortaleza', orders: 2808, revenue: 1284501 },
      { region: 'Curitiba', orders: 2774, revenue: 1247515 },
      { region: 'Sao Paulo', orders: 2740, revenue: 1247509 },
      { region: 'Brasilia', orders: 2707, revenue: 1187450 }
    ],
    monthly_evolution: [
      { month: '2026-01', revenue: 941531 },
      { month: '2026-02', revenue: 832489 },
      { month: '2026-03', revenue: 807929 },
      { month: '2026-04', revenue: 950401 },
      { month: '2026-05', revenue: 924045 },
      { month: '2026-06', revenue: 868414 },
      { month: '2026-07', revenue: 876592 },
      { month: '2026-08', revenue: 972383 },
      { month: '2026-09', revenue: 898078 },
      { month: '2026-10', revenue: 865620 }
    ],
    quality: { valid_rows: 19606, invalid_rows: 394, error_file: 'errors.csv' }
  }
}

```

Document MongoDB — agrégations et qualité

9.3 Jenkins — pipeline vert

Pipeline `ecommerce` exécuté avec succès (build #2).



Pipeline Jenkins en succès

10. Difficultés rencontrées

- **Branchement conditionnel Airflow** : combinaison du `BranchPythonOperator` avec les Trigger Rules pour que le rapport final s'exécute même lorsqu'une branche est ignorée (`skipped`).
- **Tâches dynamiques** : création des tâches d'analyse par catégorie au parsing du DAG tout en gardant un graphe stable et lisible.
- **Qualité des données** : conception de règles de rejet déterministes et isolation des lignes fautives dans un fichier d'erreurs dédié.
- **Conteneurisation** : orchestration cohérente de Postgres, MongoDB, Airflow (webserver + scheduler) et Jenkins via un unique `docker-compose.yml` .

11. Conclusion

Le projet met en œuvre un pipeline Data Engineering **complet et industrialisé** : le code est versionné (Git, branches `main` / `dev`), testé et déployé en continu (Jenkins), orchestré (Airflow) et les indicateurs sont historisés (MongoDB), le tout conteneurisé (Docker). La chaîne garantit des

indicateurs fiables, produits automatiquement et tracés à chaque exécution, répondant ainsi à la problématique initiale de modernisation de l'analyse des ventes.

Des évolutions possibles : branchement d'un véritable connecteur Olist, ajout d'un tableau de bord (Metabase/Superset) sur MongoDB, alerting sur dégradation de la qualité, et parallélisation des analyses par catégorie.